Academic Doubt Forum

Hackathon Project Blueprint & Technical Specification

OFFICIAL BLUEPRINT

Problem Statement

Students often have small academic doubts but hesitate to ask in class. They resort to messaging peers individually, causing the exact same doubt to be answered repeatedly across fragmented personal chats instead of being indexed, verified, and publicly accessible for everyone.

1. THE 7 CORE FEATURES & FALLBACK ENGINE

#	Feature Name	Technical Mechanism & Functionality
1	**Centralised Subject-Wise Q&A**	Subject taxonomy (CS, Math, Physics, Electronics). Students post open doubts; Mentors (seniors, peers, faculty) post verified answers.
2	**AI Best-Fit Solution Analyzer**	Synthesizes & evaluates submitted answers for conceptual depth, clarity, and mathematical accuracy. Automatically awards & pins the "AI Best-Fit Solution" badge.
3	**PDF Generation (Revision Notes)**	One-click client-side export compiling doubts, verified solutions, and key takeaways into a print-ready PDF revision sheet for exams.
4	**Redundancy & Deduplication Engine**	Real-time semantic matching while typing. Detects existing identical/similar doubts (>65% similarity) and links directly to existing answers to prevent forum clutter.
5	**AI Chatbot vs. Mentor Toggle**	Dual routing switch: **[Ask AI]** for instant streaming solution, or **[Ask Mentors]** to post to the community board for peer review.
6	**Kaggle-Style Feedback & Ratings**	Upvoting/downvoting and "Verified Correct" ticks. Mentors earn reputation points and tier badges (Novice → Expert → Master → Grandmaster).
7	**Voice Assistance (STT & TTS)**	Speech-to-text dictation via Web Speech API / Whisper; solution read-aloud playback powered by native browser `window.speechSynthesis`.
+	**Gemini Multi-Key Fallback Engine**	Multi-key failover manager: Automatically catches HTTP 429 (quota/rate limit) on the primary Gemini key and instantly routes to the secondary fallback key.

2. COMPLETE PROJECT DIRECTORY & CODE STRUCTURE

```
academic-doubt-forum/
├── public/
│   ├── favicon.ico # App icon
│   └── placeholder.svg # Subject category icons
├── src/
│   ├── components/
│   │   ├── Navbar.jsx # Header with subject tabs, search & voice trigger
│   │   ├── DoubtFeed.jsx # Filterable feed of academic questions
│   │   ├── QuestionCard.jsx # Doubt item, subject badge, vote counts, TTS speaker
│   │   ├── AskDoubtModal.jsx # Live redundancy checker, voice STT, AI/Mentor toggle
│   │   ├── AnswerSection.jsx # Mentor answer submission, Kaggle upvotes/downvotes
│   │   ├── BestFitAnswerCard.jsx # Feature 2: Highlighted AI top-rated solution badge
│   │   ├── RedundancyAlert.jsx # Feature 4: Real-time duplicate question suggestion banner
│   │   ├── PdfExportButton.jsx # Feature 3: One-click formatted revision sheet PDF download
│   │   ├── LeaderboardModal.jsx # Feature 6: Mentor tier leaderboard (Kaggle-style)
│   │   └── SettingsModal.jsx # Gemini Primary & Secondary fallback key manager
│   ├── lib/
│   │   ├── geminiEngine.js # Multi-key failover manager & AI prompts
│   │   ├── redundancyMatcher.js # Jaccard + Cosine token similarity deduplication engine
│   │   ├── voiceController.js # Web Speech STT recognizer & window.speechSynthesis TTS
│   │   ├── pdfGenerator.js # Vector print formatting for doubt revision sheets
│   │   ├── storage.js # LocalStorage persistence for questions, answers, ratings
│   │   └── sampleData.js # Pre-loaded academic doubts across CS, Math, Physics
│   ├── App.jsx # Main application router and state coordinator
│   └── index.css # Tailwind utility styles & print stylesheets
├── index.html # HTML5 shell with Google Font & meta tags
└── package.json # Dependencies, scripts, and build configuration
```

Architecture & Algorithmic Workflows

End-to-End Resolution Pipeline & Failover Engine

ARCHITECTURE

3. END-TO-END SYSTEM FLOW

Step 1 **Student Doubt Input:** Student selects Subject (e.g. *Data Structures*) and types or dictates question using **Voice STT**.

Step 2 **Redundancy Guard (Feature 4):** Live tokenizer computes similarity against existing doubts. If similarity > 65%, shows duplicate banner with direct solution link.

Step 3 **Mode Branching (Feature 5):**
- *Ask AI:* Gemini Engine generates immediate step-by-step conceptual breakdown.
- *Ask Mentors:* Doubt is published to the public subject board for seniors and faculty.

Step 4 **Peer Mentorship & Answering (Feature 1):** Seniors, juniors, or faculty submit answers.

Step 5 **AI Best-Fit Selection (Feature 2):** When multiple answers exist, AI analyzer ranks answers on depth and clarity → Pins "AI Best-Fit Solution".

Step 6 **Feedback & Gamification (Feature 6):** Students upvote/downvote answers. Mentors gain points and climb ranks up to Grandmaster.

Step 7 **Voice Out & PDF Export (Features 3 & 7):** One-click voice read-aloud via `window.speechSynthesis` or instant revision PDF export.

4. ALGORITHMIC DEEP DIVES

Redundancy Detection Algorithm

Feature 4

Token Jaccard & Cosine Similarity Matcher

```
function calculateSimilarity(q1, q2) {
  const t1 = tokenize(q1.toLowerCase());
  const t2 = tokenize(q2.toLowerCase());
  const inter = t1.filter(x ⇒ t2.includes(x));
  const union = new Set([ ... t1,  ... t2]);
  const jaccard = inter.length / union.size;
  return Math.round(jaccard * 100);
}
// Score ≥ 65%: Flag as duplicate thread
```

Kaggle Rating & Tier Rules

Feature 6

Reputation Score Calculation Formula

```
// Formula:
Score = (Upvotes * 10) − (Downvotes * 5)
        + (BestFitAwards * 30)

Badge Tiers:
0 − 49 pts    : Novice 🥉
50 − 199 pts  : Expert 🥈
200 − 499 pts : Master 🥇
500+ pts      : Grandmaster 👑
```

5. GEMINI API MULTI-KEY FALLBACK ENGINE

Failover Guarantee: If Primary Gemini Key hits HTTP 429 (quota exhausted), engine catches the error, switches the active key pointer to Secondary Key, and retries seamlessly. If all keys fail, smart offline heuristics engage.

```
class GeminiFallbackEngine {
  constructor(primaryKey, secondaryKey) {
    this.keys = [primaryKey, secondaryKey].filter(Boolean);
    this.activeKeyIndex = 0;
  }

  async executeWithFallback(promptPayload) {
    while (this.activeKeyIndex < this.keys.length) {
      const currentKey = this.keys[this.activeKeyIndex];
      try {
        return await fetchGemini(promptPayload, currentKey);
      } catch (err) {
        if (err.status ═══ 429 || err.message.includes("QUOTA")) {
          console.warn(`Key exhausted! Failover to secondary key ... `);
          this.activeKeyIndex++;
        } else { throw err; }
      }
    }
    return offlineHeuristicAnalyzer(promptPayload); // Safety net
  }
}
```

Data Models & Hackathon Pitch Script

Database Schemas, Voice API Implementation, & Presentation Flow

DEMO GUIDE

6. DATA MODELS & JSON SCHEMAS

Question Schema

Question Entity

```
{
  "id": "q_101",
  "subject": "Computer Science",
  "title": "Dijkstra vs A* Algorithm differences?",
  "author": { "name": "Priya Sharma", "role": "Student"
},
  "isResolved": true,
  "bestFitAnswerId": "ans_301",
  "mode": "mentor",
  "createdAt": "2026-09-08T10:30:00Z"
}
```

Answer & Rating Schema

Answer Entity

```
{
  "id": "ans_301",
  "questionId": "q_101",
  "author": { "name": "Rahul Verma", "tier": "Master" },
  "content": "Dijkstra expands uniformly; A* uses
heuristics ... ",
  "upvotes": 18,
  "downvotes": 1,
  "isBestFit": true,
  "isVerified": true
}
```

7. VOICE ASSISTANCE INTEGRATION (STT & TTS)

Voice Input (Speech-to-Text)

Web Speech API

```
const rec = new webkitSpeechRecognition();
rec.lang = 'en-US';
rec.onresult = (e) => {
  const text = e.results[0][0].transcript;
  doubtInput.value = text;
  triggerRedundancyCheck(text);
};
rec.start();
```

Voice Output (Text-to-Speech)

SpeechSynthesis

```
function speakAnswer(text) {
  if (!'speechSynthesis' in window) return;
  window.speechSynthesis.cancel();
  const utter = new SpeechSynthesisUtterance(text);
  utter.rate = 1.0;
  window.speechSynthesis.speak(utter);
}
```

8. 3-MINUTE HACKATHON PITCH SCRIPT & JUDGE WALKTHROUGH

Time	Demo Screen	Talking Points for Hackathon Judges
0:00 - 0:40	**Problem & Voice Input**	*"Judges, students hesitate to ask questions in class, and the same doubt is answered 50 times in private chats. Watch: I hit the mic, speak my doubt: 'Explain time complexity of Quicksort', and speech-to-text fills it instantly."*
0:40 - 1:15	**Feature 4: Redundancy Guard**	*"Notice the live banner pop up: 'Similar solved question found: Quicksort average vs worst case'. One click takes me to the pre-existing solution—preventing duplicate forum bloat!"*
1:15 - 1:50	**Feature 5 & Feature 2 (AI Best-Fit)**	*"Students can toggle 'Ask AI' for instant unblocking or post to mentors. When seniors submit multiple explanations, our AI Analyzer identifies the most rigorous response and pins the AI Best-Fit badge."*
1:50 - 2:30	**Kaggle Ratings & Fallback Engine**	*"Upvoting answers levels up mentors from Novice to Grandmaster. And even if our primary Gemini key hits API rate limits, our automated fallback engine switches keys instantly with zero failure."*
2:30 - 3:00	**PDF Export & Voice Out**	*"Finally, with one click, the student downloads a revision note PDF for exam prep, or clicks the speaker icon to listen to the verified answer hands-free!"*